CSA0613 DESIGN AND ANALYSIS OF ALGORITHMS

CO3_ASSESSMENT TOOL 2

Q26. Case Study: Airline Reservation Sorting System (Merge Sort)

An airline reservation system sorts flight bookings based on departure time and priority. Analyze how Merge Sort ensures stable and efficient sorting. Identify issues such as large data volume and real-time updates. Apply divide and conquer techniques to explain the process. Analyze time complexity and design a solution, justifying its effectiveness.

Aim

To implement Merge Sort for sorting airline flight bookings based on departure time and analyze its efficiency using the Divide and Conquer technique.

Problem Statement

An airline reservation system sorts flight bookings based on departure time and passenger priority. The system must efficiently handle large numbers of bookings while maintaining the order of bookings with the same departure time. Merge Sort is used because it is a stable and efficient sorting algorithm.

Objective

- To understand the working of Merge Sort.
- To apply the Divide and Conquer approach.
- To sort flight bookings based on departure time.
- To analyze the time and space complexity.
- To justify why Merge Sort is suitable for airline reservation systems.

Algorithm

Step 1: Read the number of flight bookings.

Step 2: Store the Flight Number and Departure Time.

Step 3: Divide the booking list into two equal halves.

Step 4: Recursively sort each half.

Step 5: Merge the two sorted halves.

Step 6: Display the sorted flight bookings.

Python program

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i][1] <= right[j][1]:
                arr[k] = left[i]
            else:
                arr[k] = right[j]
                j += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
n = int(input("Enter number of flights: "))
flights = []
for i in range(n):
    f = input("Flight Number: ")
    t = int(input("Departure Time: "))
    flights.append((f, t))
merge_sort(flights)
print("\nSorted Flight Bookings:")
for f, t in flights:
    print(f, "-", t)
```

Output

```
Enter number of flights: 5
Flight Number: AI101
Departure Time: 1500
Flight Number: AI102
Departure Time: 900
Flight Number: AI103
Departure Time: 1800
Flight Number: AI104
Departure Time: 1100
Flight Number: AI105
Departure Time: 1200

Sorted Flight Bookings:
AI102 - 900
AI104 - 1100
AI105 - 1200
AI101 - 1500
AI103 - 1800

=== Code Execution Successful ===
```

Time Complexity

Case	Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)

Space Complexity

O(n)

Advantages

- Stable sorting algorithm.
- Efficient for large datasets.
- Predictable performance.
- Suitable for external sorting.
- Uses Divide and Conquer strategy.

Limitations

- Requires extra memory.
- Not an in-place sorting algorithm.
- Less suitable for frequent real-time insertions without additional optimization.

Result

The Airline Reservation Sorting System was successfully implemented using Merge Sort. The algorithm efficiently sorted flight bookings based on departure time while maintaining stability. With a time complexity of **O(n log n)**, Merge Sort is an effective solution for handling large-scale airline reservation data.